

Leave Request — How It Works

A plain-English guide to the Leave Request feature in the Employee Attendance mobile app.

1. What is Leave Request?

Leave Request is the employee-facing flow for applying for time off — sick leave, casual leave, annual leave, personal leave, or emergency leave. The app collects the leave type, date range (or half-day flag), and a written reason, then submits it to a manager-approval workflow on the Odoo backend.

What it produces:

- One `hr.leave` (or the custom-equivalent model in the addon) record per request.
- A state machine: **Draft** → **Pending** → **Approved / Rejected / Cancelled**.
- Email / in-app notification to the manager when a new request lands in their inbox.

The employee can apply from anywhere, anytime — there's no GPS check, no camera, no shift-time gate.

2. The five leave types

Each type is colour-coded in the app to make scanning the history easy.

Type	Use case	Colour	Icon
Sick Leave	Medical reasons. May require a doctor's note attachment in some deployments.	Red	local-hospital
Casual Leave	Short personal time (errand, half-day appointment, family obligation).	Orange	event-available
Annual Leave	Planned vacation, counts against yearly entitlement.	Blue	beach-access
Personal Leave	Other personal reasons that don't fit Sick or Casual (e.g. wedding, family function).	Purple	person
Emergency Leave	Unforeseen urgent situations (e.g. family emergency, sudden travel).	Bright red	warning

The label and colour are mapped via the `LEAVE_TYPES` constant in `UserAttendanceScreen.js`. Each type maps to a string code (`'sick'` , `'casual'` , etc.) which is what the server stores.

3. The form

The Leave Request form has just these fields:

- **Type** — radio-card selector showing all five types with their colours and icons.
- **From Date** — date picker, defaults to today.
- **To Date** — date picker, optional. If left empty, the request is treated as a single-day leave on `From Date`.
- **Half Day** — toggle. When ON, the `To Date` is hidden and the request becomes a 0.5-day leave on `From Date`.
- **Reason** — free-text field, **required**. The submission is blocked if this is empty.

Once the user has filled the form, they tap **Submit** at the bottom.

4. The submit flow

1. Validate the **Reason** field is non-empty. If blank, a toast appears: *"Please enter a reason for leave"*.

2. Format dates for Odoo as `YYYY-MM-DD`.
3. Resolve the type label (`'casual'` → `'Casual Leave'`).
4. Show a confirmation alert summarising the request:

```
Type: Casual Leave
From: 27 May 2026
To: 28 May 2026

Reason: Family wedding in Chennai
```

Buttons: **Submit** / **Cancel**.

5. On tap **Submit**:
 - Loading spinner shown.
 - Call `submitLeaveRequest(uid, type, fromStr, toStr, reasonStr, empId, isHalfDay)`.
 - On success: toast "*Leave request submitted for approval!*", clear form, switch to the **History** tab so the user sees their fresh request at the top of the list.
 - On failure: alert with the server's error message.

The submission goes to the offline queue if the device is offline, just like attendance check-ins.

5. The history tab

Below the form, the screen has a tab strip with two tabs:

- **Form** — the input view described above.
- **History** — chronological list of every leave request the user has submitted.

Each history row shows:

- The leave type label and colour chip.
- Date range (or single date / half-day indicator).
- Current state — **Draft**, **Pending**, **Approved**, **Rejected**, or **Cancelled** — with a coloured pill.
- A small reason snippet (truncated).
- A **Cancel** button on rows that are still in *Draft* or *Pending* state.

Tapping a row expands it to show the full reason and any manager note / rejection reason.

The list is fetched via `fetchLeaveHistory()` on tab entry. Pull-to-refresh re-fetches.

6. Cancelling a request

A pending request can be cancelled by the employee before the manager acts on it.

Flow:

1. User taps **Cancel** on a row in History.
2. Confirmation alert: "*Cancel this leave request?*" with **Yes** / **No** (destructive).
3. On **Yes**: app calls `cancelLeaveRequest(requestId)`, the state changes to **Cancelled**, toast confirms.
4. The history list is refreshed automatically.

Cancelled requests stay in the history list as audit trail — they aren't deleted.

Once a manager has approved or rejected the request, the employee can no longer cancel from the app. They'd need to ask their manager to revert the state via the Odoo back-office.

7. State lifecycle

A leave request moves through this state machine:

```
Draft → Pending → Approved → (Leave taken)
                        → Rejected
                        → Cancelled
```

- **Draft** — exists briefly before submission, rarely seen by the employee.
- **Pending** — submitted, awaiting manager approval. Default state right after submit.
- **Approved** — manager (or HR) approved. Affects the company's leave balance / shift roster.
- **Rejected** — manager declined. The manager's note appears on the history row.
- **Cancelled** — employee cancelled before approval, or HR cancelled post-approval (rare).

The state pill in the UI uses these labels via `getLeaveStateLabel()`.

8. Half-day leaves

The **Half Day** toggle on the form is a one-tap shortcut for a 0.5-day leave.

When toggled ON:

- The **To Date** picker hides — it's implicitly the same day as **From Date**.
- The server stores `is_half_day = true` and `duration = 0.5`.
- The history row shows "(Half day)" next to the date.

When toggled OFF (default):

- The request is full days.
- If **To Date** is empty, it's a single full day.
- If **To Date** is set, the duration is `(To Date - From Date + 1)` calendar days.

The half-day check happens server-side too, so a half-day request can't be silently bumped to a full day if the toggle was wrong.

9. Notifications and manager flow

When a request is submitted:

- Odoo creates a discussion-thread message on the leave record, mentioning the manager (`message_post` with `partner_ids = [manager.partner_id.id]`).
- The manager gets an email and/or in-app notification.
- The manager opens the leave record in Odoo (back-office), reviews the reason and dates, and clicks **Approve** or **Refuse**.
- An email goes back to the employee with the decision and the manager's note (if any).
- The mobile app's history list reflects the new state on next refresh.

The mobile app does NOT have a manager-side approval screen — managers use the Odoo web back-office for that. Employees only see the read-only history of their own requests.

10. Offline support

Leave Request works the same offline-first way as check-ins:

- Submission while offline → goes to the offline queue with the full payload.
- The local state shows the request in History with a small **"Pending sync"** indicator.
- When the device reconnects, `OfflineSyncService` replays the queue.
- On success, the local indicator clears and the request is now a real `hr.leave` record on the server.

- The history is fetched fresh on next connection so any state changes (manager approval that happened while the user was offline) are reflected.

11. Validation rules

The form enforces these rules client-side:

- **Reason** must be non-empty after trim. Otherwise toast *"Please enter a reason for leave"*.
- **From Date** must be set (defaults to today, picker always returns a value).
- **To Date**, if set, must be $\geq$ From Date. If the user picks an earlier date, the picker UI prevents it (`minimumDate = leaveFromDate`).
- **Half Day** toggle automatically clears To Date when turned ON.

Server-side rules (beyond the app's reach):

- Annual leave can't exceed the configured yearly entitlement.
 - Overlapping leaves are rejected.
 - Some types may require attachments (deployment-specific).
 - Some types may auto-approve below a certain duration (e.g. 0.5 days of sick leave auto-approves).
-

12. Where the data lives (UI → Odoo)

What you see in the app	Where it lives on the server
Leave request	<code>hr.leave</code> (or the addon's custom model)
Leave type	<code>hr.leave.type</code> with the code mapped (<code>sick</code> , <code>casual</code> , <code>annual</code> , <code>personal</code> , <code>emergency</code>)
State (Draft / Pending / Approved / Rejected / Cancelled)	<code>hr.leave.state</code> field
Manager approval thread	<code>mail.thread</code> messages on the leave record
Employee	<code>hr.employee</code>
Manager	<code>hr.employee.parent_id</code> (the leave is mentioned to this user's partner)

13. Buttons and gates — quick cheat sheet

Button	Visible when
Form tab	Always (default tab on entry).
History tab	Always; auto-switches to here after a successful submission.
Submit	On the Form tab; tap-able after reason is filled.
Cancel (on a row)	The row's state is Pending or Draft .
Half Day toggle	On the Form tab; clears To Date when toggled ON.

14. A worked example

It's Sunday evening, 9:00 PM. Ravi wants to take Friday off for a family wedding in Chennai.

1. Open the app → tap **Leave Request** from the mode picker.
2. Verify with fingerprint or PIN.
3. Form opens. He taps the **Casual Leave** card. The card highlights orange.
4. **From Date** = next Friday (29 May 2026).
5. **To Date** = same Friday — he leaves it as the default, so it'll be a single-day request.
6. **Half Day** = OFF.
7. **Reason** = *"Family wedding in Chennai. Will return Monday morning."*
8. Tap **Submit**.
9. Alert: *"Type: Casual Leave / From: 29 May 2026 / (Single day) / Reason: Family wedding in Chennai. Will return Monday morning."* He taps **Submit**.
10. Toast: *"Leave request submitted for approval!"* The form clears and the screen switches to the **History** tab. His new request sits at the top, state **Pending**, in orange.
11. Two hours later his manager opens Odoo, approves the request. An email goes to Ravi.
12. Next time Ravi opens the app and looks at his history, the same row now shows state **Approved** in green.

That's the full Leave Request cycle.